8-4 · Streamlit 기반의 챗봇 인터페이스 개발 · 정리·회고 0.5H

# 정리·회고

## 챗봇 구조를 한 표로, 8-5 최종일 시나리오 예고

이 시간의 목표

# 30분 뒤에는

① 챗봇 뼈대를 「하려는 일 ↔ 코드」 한 표로 정리한다

② 오늘의 체크리스트로 빠진 단계를 스스로 찾는다

③ 자주 겪는 오류 네 가지의 원인과 해결을 안다

④ 8-5 최종일에 필요한 것이 켜져 있는지 확인한다

**오늘 파일을 지우지 마세요 — 8-5 의 출발점입니다**

오늘의 요약

# 하려는 일 ↔ 코드 — 여덟 줄로 정리

| 하려는 일 | 코드 |
| --- | --- |
| 이력 저장소 만들기 | if "messages" not in st.session_state: … |
| 이력 다시 그리기 | for m in st.session_state.messages: … |
| 입력 받기 | st.chat_input("…") |
| 말풍선 | with st.chat_message("user"): st.markdown(…) |
| 연결 캐시 | @st.cache_resource |
| 값 캐시 | @st.cache_data |
| 즉시 재실행 | st.rerun() |
| 접는 영역 | with st.expander("근거"): … |

**사이드바는 with st.sidebar: … · 실행은 streamlit run app.py**

오늘의 체크리스트

# 열네 개를 여덟 줄로 — 스스로 확인해 보세요

1. 상호작용마다 스크립트가 다시 실행되는 것을 화면으로 확인했다
2. 일반 변수와 session_state 의 차이를 직접 눌러 봤다
3. 다시 그리는 for 루프를 지워 대화가 사라지는 것을 재현했다
4. chat_input / chat_message 로 말풍선을 구분해 그렸다
5. 사이드바·대화 지우기·@st.cache_resource 효과를 확인했다
6. 화면을 붙이기 전에 검색만 따로 시험했다
7. 임베딩 → query_points → LLM 호출 3단계를 코드에서 짚는다
8. 기준값 0.4 로 김치찌개 질문이 LLM 에 닿지 않고 막혔다

자주 겪는 오류

# 증상으로 찾는 원인과 해결 — 네 가지

| 증상 | 원인 | 해결 |
| --- | --- | --- |
| **메시지를 보내면<br>이전 대화가 사라짐** | 일반 변수에 담았거나<br>다시 그리는 루프 없음 | 이력은 session_state 에<br>상단 루프로 전체 다시 그림 |
| **Qdrant 연결 거부<br>connection refused** | 8-3 .env 의 QDRANT_URL=<br>http://qdrant:6333 을 복사 | echo $QDRANT_URL 이<br>비어 있는지 확인 |
| **Vector dimension error<br>expected 768, got 1536** | 7주차와 다른 모델 경로로<br>질문을 임베딩 | PROVIDER 를 7주차와 같게<br>.env 에 LLM_PROVIDER 고정 |
| **코드를 고쳤는데<br>화면이 그대로** | 캐시된 결과가 남았거나<br>브라우저가 옛 화면 | 우측 상단 Clear cache<br>또는 streamlit 재시작 |

**docker ps 에서 qdrant 가 Up · 0.0.0.0:6333 인지 함께 본다 · PROVIDER 는 python rag_env.py 로**

예상 질문

# 자주 나오는 질문 네 가지

1. **여러 사람이 동시에 쓰면 대화가 섞이나요?**
   섞이지 않는다 — session_state 는 브라우저 세션 단위. 다만 서버 자원은 함께 쓴다

2. **대화를 영구 보관하려면?**
   DB 에 저장한다 — session_state 는 서버 재시작에 사라진다

3. **배포는 어떻게 하나요?**
   Streamlit Community Cloud, 또는 Docker 이미지로 만들어 서버에 — 이번 과정 범위 밖

4. **응답이 느린데요?**
   임베딩·검색·생성 세 호출이 순차로 — 스트리밍으로 체감 속도를 올리는 것이 가장 효과적

8-5 준비

# 8-5 까지 켜 두고 남겨 둘 것

1. **오늘 파일을 지우지 않는다**
   app.py 가 8-5 최종본의 출발점 — cd ~/data-eng/week8 && ls

2. **docker ps --format "table {{.Names}}\t{{.Status}}"**
   ollama · qdrant · rag-minio 가 Up 인지

3. **app.py 가 있고 세 컨테이너가 Up 이면**
   8-5 준비 완료 — Airflow 는 8-5 실습점검에서 다시 켠다

8-5 할 일

**최종본 다듬기**

**시연·제출**

> **8-5: 하이브리드 검색·출처 표시·설정 화면으로 다듬기 → 전 주차를 잇는 시연 시나리오 → 최종 산출물(DAG + 챗봇 앱 + 문서) 제출**

다음 회차 예고

# 8-5 · 최종 RAG 챗봇 개발 — 파이프라인 통합

01
### 이론 1H
최종 아키텍처
점검

02
### 실습 1.5H
DAG + RAG 챗봇
통합 구축

03
### 실습 1H
엔드투엔드
검증·시연

04
### 정리 0.5H
과정 회고 ·
산출물 제출 확인

**준비: 8-5 실습점검에서 Airflow 일곱 서비스 · ollama · qdrant · rag-minio · lecture_kb 11 조각 · 8-4 가상환경을 한 번에 확인**

8-4 · 정리·회고 0.5H 마무리

# 다음 회차는 8-5 최종 RAG 챗봇

8주 전 구간(수집→Lake→변환→VectorDB→챗봇)을 하나의 시스템으로 묶고, 새 문서가 답변에 반영되는 시나리오로 검증합니다

- ✓ **8주 전 구간을 한 아키텍처 그림으로**
- ✓ **스케줄·재시도·지운 문서 정리·적재 시각**
- ✓ **새 문서 → DAG 실행 → 챗봇 답변 반영 검증**
- ✓ **코드·README·requirements·.env.example 패키징**

준비할 것

**세 컨테이너 Up · app.py 그대로**